The Competence Trap

Why the most experienced developers are the most resistant to the tool that was always going to exist

An essay by Juan Carlos Ghiringhelli

Someone on X posted an observation last week that I haven't been able to shake: junior developers are embracing AI-assisted development with enthusiasm, very senior engineers understand immediately why it matters, and the cohort in the middle — the twenty-year veterans, the staff engineers, the technical leads who built their careers on mastery — are the ones digging in.

The pattern is counterintuitive until you think about it for thirty seconds. Then it's inevitable.

Junior developers have no sunk cost. They entered a world where the tool already existed. They didn't spend a decade internalizing the syntax, the patterns, the framework idiosyncrasies, the hard-won debugging reflexes that constitute expertise in the execution layer of software development. They never had to. The tool was there. Embracing it isn't a betrayal of anything they built. It's just the environment.

Very senior engineers — the ones who've been in the field long enough to have worked on systems that nobody maintains anymore, who've watched the same tech debt accumulate across enough companies to know it will never be addressed through normal means — understand something the middle cohort doesn't: most of the code that exists right now is quietly rotting. Not dramatically. Not in any way that will trigger an emergency. Just slowly, session by session, deadline by deadline, becoming harder to reason about, harder to change, more expensive to touch. These engineers see AI-assisted development not as a threat to their expertise but as the first credible answer to a problem they've watched worsen for twenty years.

The middle is where it gets complicated.

The twenty-year veteran has something the junior doesn't and something the thirty-year veteran has already recategorized: a very large investment in execution skill. Not a theoretical investment. A real one. Years of deliberate practice, late nights with documentation, hard problems solved through accumulated pattern recognition. That investment is not imaginary. It produced real results. It was the thing that got them promoted, that earned them credibility, that made them the person people came to with hard problems.

The sunk cost fallacy, in its clinical form, is using the cost of a past investment to justify a present decision. The trap here isn't that the investment was worthless. It's that it was genuinely valuable — in a context that is changing. The calculation isn't "were the past two decades worth it." It's "does the past two decades' return depend on the next decade looking like it."

For most mid-senior engineers, the honest answer is: somewhat. The deep structural knowledge — system design, domain reasoning, the ability to recognize when something is wrong before it breaks — transfers. The execution skill — knowing exactly which parameters a function takes, the specific incantation for a build configuration, the muscle memory of a particular framework — is exactly the layer the tool handles now.

That distinction is where people get stuck. Because the execution skill was the visible, measurable, teachable part. It was the thing that could be demonstrated in an interview, evaluated in a code review, cited in a performance review. The structural judgment underneath it is harder to name and harder to claim.

Here is what I should say about my own position: I should be in the resistant middle. By every demographic indicator — career stage, years of accumulated practice, professional identity built on technical depth — I should be the person finding reasons why the tool doesn't really work, why the outputs aren't trustworthy, why real expertise still requires doing it by hand.

I'm not. I built the methodology that treats the tool as the primary executor and the specification as the primary artifact. I am, by any reasonable measure, one of the more committed adopters in the field.

The reason isn't that I don't have sunk costs. I do. The reason is that I got far enough into the practice to see what the tool actually does to the work: it doesn't replace the judgment. It removes the friction between the judgment and its execution. That's not a threat to expertise. It's what expertise was always supposed to feel like.

The junior developer gets there by default. The very senior engineer gets there through pattern recognition. The middle cohort has to choose it — which means accepting that the most valuable part of what they built over twenty years is not the part that's changing.

That's harder than it sounds. But it's the only move that works.

Juan Carlos Ghiringhelli is the founder of Pragmaworks, the author of Generative Specification, and a twenty-year veteran who chose the other path.